

# Stress-testing the `tanglepack` numerics layer

What the manifold-growth engine can and cannot do, with evidence

`tanglepack` numerics stress campaign

2026-10-07

This report presents a 59.8-minute stress campaign of 349 isolated runs (ten experiments, E1–E10) against `tanglepack.numerics`, the layer that grows the stable and unstable manifolds of saddle orbits of area-preserving maps, detects their crossings and builds bridges. Each section follows the same pattern: a claim, the figure or table that supports it, an interpretation, and a caveat. Negative results count as findings. Every measured number in the text is generated from the raw result files (`numbers.tex`, `numbers_extra.tex`); none is typed by hand.

Branch `numerics-stress`, library commit `e963893`. The library itself was not modified.

# Contents

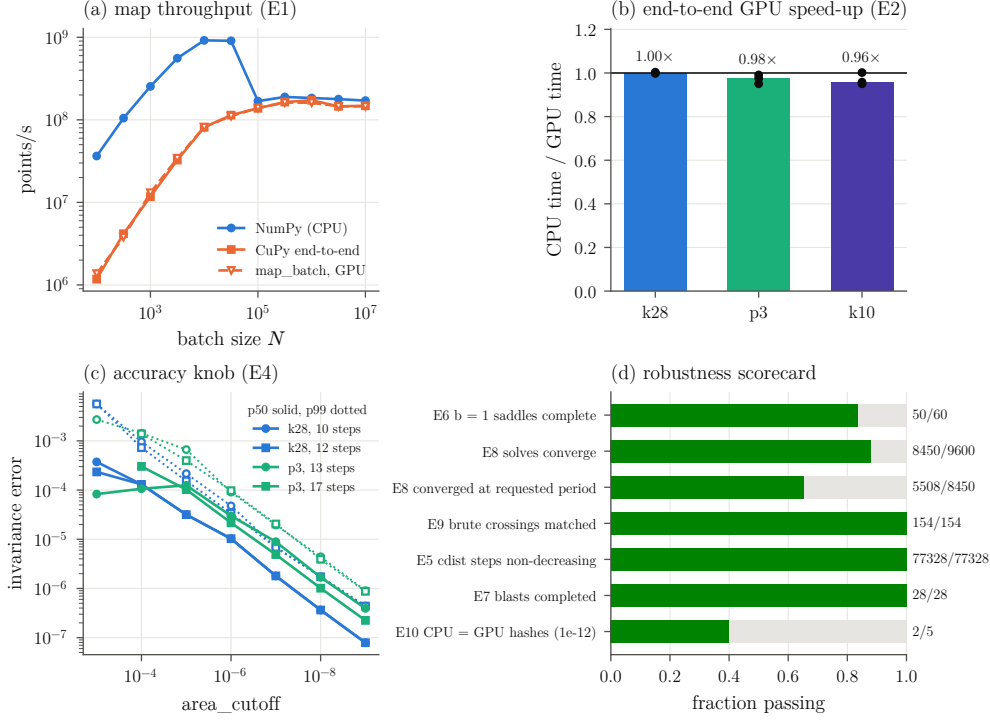
|           |                                               |           |
|-----------|-----------------------------------------------|-----------|
| <b>1</b>  | <b>Executive summary</b>                      | <b>1</b>  |
| <b>2</b>  | <b>Setup and methods</b>                      | <b>3</b>  |
| <b>3</b>  | <b>Performance and scaling (E3)</b>           | <b>4</b>  |
| <b>4</b>  | <b>GPU acceleration (E1, E2)</b>              | <b>6</b>  |
| 4.1       | Raw throughput . . . . .                      | 6         |
| 4.2       | End to end . . . . .                          | 6         |
| 4.3       | Why: Amdahl's law . . . . .                   | 6         |
| <b>5</b>  | <b>Accuracy versus cost (E4)</b>              | <b>8</b>  |
| <b>6</b>  | <b>Physical invariants (E5)</b>               | <b>9</b>  |
| <b>7</b>  | <b>Correctness of crossing detection (E9)</b> | <b>10</b> |
| <b>8</b>  | <b>Robustness across parameters (E6)</b>      | <b>11</b> |
| <b>9</b>  | <b>Failure modes and breaking points (E7)</b> | <b>13</b> |
| 9.1       | Escape to infinity . . . . .                  | 13        |
| 9.2       | Depth . . . . .                               | 13        |
| 9.3       | Blasts . . . . .                              | 14        |
| 9.4       | Near tangency: artefact crossings . . . . .   | 15        |
| <b>10</b> | <b>Fixed-point solver (E8)</b>                | <b>15</b> |
| <b>11</b> | <b>Reproducibility (E10)</b>                  | <b>16</b> |
| <b>12</b> | <b>Failure taxonomy</b>                       | <b>18</b> |
| <b>13</b> | <b>Recommendations</b>                        | <b>19</b> |
| <b>14</b> | <b>Threats to validity</b>                    | <b>20</b> |
| <b>A</b>  | <b>Tables</b>                                 | <b>21</b> |
| <b>B</b>  | <b>Reproduction</b>                           | <b>22</b> |
| <b>C</b>  | <b>Configuration and outcome counts</b>       | <b>23</b> |

# 1 Executive summary

## Five headline findings.

1. **The GPU does not help, and cannot help in the current design.** The end-to-end speed-up is  $0.999\times$  (k28),  $0.975\times$  (p3) and  $0.958\times$  (k10). The map takes at most 0.84% of growth time, so Amdahl’s law caps the gain at  $1.008\times$  (Section 4).
2. **Cost is linear in the point count, but the point count is not set by the dynamics.** Step time scales as  $N^{1.06}$  and memory as 1 058 B/point. However, once the unstable arm escapes to infinity a single step multiplies the points by up to 61.8, against  $\lambda_u = 7.15$  (Section 3).
3. **area\_cutoff is a well-behaved accuracy knob.** The invariance error falls as  $\text{cutoff}^{0.70}$  and the point count grows as  $\text{cutoff}^{-0.332}$  (k28), close to the  $2/3$  and  $-1/3$  that chord geometry predicts. At  $10^{-7}$  the median invariance error on k28 is  $1.8 \times 10^{-6}$  for 0.107s of pipeline time. The library default,  $10^{-4}$ , is too coarse for the period-3 case: it invents and drops crossings (Section 5).
4. **Transverse crossings are found exactly, and the physical invariants hold.** An independent brute force matches all 154/154 crossings with 0 missed and 0 extra. Lobe areas are preserved to  $5.0 \times 10^{-3}$ , and 0 cdist decreases occur in 77 328 steps (Sections 6 and 7).
5. **Two failures are silent.** (a) Near the saddle-centre bifurcation the number of registered crossings at  $k + 1 = 10^{-4}$  grows from 25 to 499 as the resolution is refined, with 0 warnings. These are artefacts of the polygons, not dynamics. (b) The fixed-point solver hands back lower-period orbits: 1 424 of them were accepted as saddles of the requested period (Sections 9 and 10).

Figure 1 puts one panel per theme side by side, and Table 1 grades each capability with a pointer to its evidence. Section 12 catalogues the failures, and Section 13 expands the recommendations below.



**Figure 1:** Campaign overview. (a) Map throughput: NumPy beats the end-to-end CuPy path at every batch size, and the `map_batch` GPU path only catches up at the largest batches (Section 4). (b) End-to-end speed-up per case: every bar is at or below 1. (c) Invariance error falls steadily as `area_cutoff` is refined; solid is the median, dotted the 99th percentile. (d) Pass fractions of the main checks. The short bars are the solver’s requested-period rate and the CPU/GPU hash agreement; both are explained in Sections 10 and 11.

**Table 1:** Scorecard. Status: **works** = does what it should over the tested range; **limited** = works with a documented restriction; **fails** = does not deliver.

| capability                                    | status         | evidence                                                                  |
|-----------------------------------------------|----------------|---------------------------------------------------------------------------|
| Manifold growth, stop-condition driven        | <b>works</b>   | escape case stops at 478 points (Section 9.1)                             |
| Manifold growth, fixed step count             | <b>fails</b>   | killed at the 20 GB cap after 9 steps (Section 9.1)                       |
| Cost scaling                                  | <b>works</b>   | $\text{time} \propto N^{1.06}$ , 1 058 B/point (Section 3)                |
| Accuracy control ( <code>area_cutoff</code> ) | <b>works</b>   | $\text{error} \propto \text{cutoff}^{0.70}$ (Section 5)                   |
| Transverse crossing detection                 | <b>works</b>   | 154/154 matched, 0 extra (Section 7)                                      |
| Crossings near tangency                       | <b>fails</b>   | count tracks resolution, 0 warnings (Section 9.4)                         |
| Self-crossing detection                       | <b>limited</b> | 4 $u \times u$ crossings exist, none reported (Section 7)                 |
| Area preservation, invariants                 | <b>works</b>   | lobe $ A'/A - 1  \leq 5.0 \times 10^{-3}$ (Section 6)                     |
| Arrangement <code>image_of</code>             | <b>limited</b> | 47 images found, 39 missing (Section 6)                                   |
| Blasting                                      | <b>limited</b> | 10 blasts run, topology unreliable after them on k28 and p3 (Section 9.3) |
| Inversion saddle                              | <b>limited</b> | topology asserts at 4 and 5 steps; OOM at 7 and 8 (Section 9.2)           |
| Parameter robustness ( $b = 1$ saddle)        | <b>limited</b> | 83.3% complete; 26 trivial tangles (Section 8)                            |
| Fixed-point solver                            | <b>limited</b> | converges 88%, but accepts lower periods (Section 10)                     |
| GPU acceleration                              | <b>fails</b>   | $0.958 \times - 0.999 \times$ end to end (Section 4)                      |
| Determinism                                   | <b>works</b>   | CPU-CPU and GPU-GPU bit-identical (Section 11)                            |

What I would do next, in priority order.

1. **Guard the crossing registration near tangency.** Warn when a crossing’s  $|\sin \theta|$  is not large

compared to the local invariance error, and add a cheap  $u \times u$  self-crossing check as a resolution diagnostic. Both failures are currently silent.

2. **Make the solver check the minimal period** before `construct_fixed_point` accepts an orbit.
3. **Put a point/memory budget into growth.** Predict the next step’s points from the last growth factor, as the E3 harness already does, and make stop-condition growth the documented default.
4. **Raise the default `area_cutoff` to  $10^{-7}$ ,** which is what the k28, p3 and nested test cases already use.
5. **Leave the GPU off.** Revisit it only if refinement and the linked list move to arrays that live on the device.

## 2 Setup and methods

**The map and the pipeline.** All runs use the Hénon family (parameters  $k, b$ ;  $b = 1$  is area preserving) defined in the module `tanglepack.examples.henon`, via the frozen recipes of `tests/cases.py`. These recipes are: k10, the  $k = 10$  binary horseshoe at the default cut-off  $10^{-4}$ ; k28,  $k = 2.8$  at  $10^{-7}$ , optionally blasted; p3, the period-3 orbit of the nested map; nested, period 1 plus period 3; and inversion, the other saddle of  $k = 10$ , with both eigenvalues negative. The pipeline under test is **TangleWorkbench**: find the fixed point, then initialise the fundamental segments, grow the manifolds (`grow_n_times`, `grow_until_*`), and finally run `compute_intersections`, `trim`, `create_bridges` and the iterate table. Where an experiment touches topology (E5, E7), it is only as a downstream consumer of the numerics.

**Harness.** `stress/numerics/harness.py` runs every configuration in a fresh subprocess with a hard timeout. A psutil watchdog samples the worker’s resident set every 50 ms and kills the worker above an RSS cap. `RLIMIT_AS` is not used because CUDA reserves huge virtual ranges. Each configuration writes one JSON line with its parameters, per-stage wall times (with a device synchronise before the clock on the GPU), per-step checkpoints, metrics, captured logging warnings by template, peak RSS and git SHA. The outcome is one of `ok`, `exception`, `invariant_assert`, `oom`, `timeout` or `crash`. *Failures are data*: a sweep never stops on one, and the per-step checkpoints keep everything up to the failure. The per-experiment caps are in Table 10. Timing experiments (E1–E4, E10) ran serially with the GPU never shared; the robustness sweeps (E5–E9) ran in worker pools.

**Hardware and software.** One host (pop-os): Intel Core i9-14900K (24 cores, 8P + 16E, 32 threads, 32 MB L2, 36 MB L3), 125 GB RAM; NVIDIA GeForce RTX 5060 Ti, 16 GB, driver 580.173.02; Linux 7.1.1. Python 3.10.12, NumPy 2.2.5, SciPy 1.15.1, CuPy 14.1.1, rtree 1.4.0. Every record carries library commit e963893. The full campaign took 59.8 min of wall-clock, after a 1.2-minute `--quick` pilot.

### Experiments.

#### E1 Raw map throughput

NumPy against CuPy (end to end, compute only, transfer only) and the real `map_batch` path,  $N = 10^2$ – $10^7$ .

#### E2 End-to-end GPU

k28, p3 and k10, CPU against GPU, three repeats; also a cProfile breakdown, a sweep of `min_batch_points` and a lockstep CPU/GPU comparison.

### E3 Scaling

Growth step by step to  $\sim 2 \times 10^6$  points for  $k \in \{2.8, 4, 6, 10\}$ , and downstream stages as a function of the segment count.

### E4 Accuracy against cost

area\_cutoff  $10^{-3}$ – $10^{-9}$  on k28 and p3 at two depths, against the finest run.

### E5 Invariants

Fixed point, manifold invariance, iterate links, lobe areas, cdist monotonicity and `image_of`, on every frozen case.

### E6 Robustness

One fixed recipe over  $k \in [0.3, 15]$  for  $b = 1$  (both period-1 points),  $b = -1$  and dissipative  $b$ .

### E7 Breaking it

Escape to infinity, 0–10 blasts, deep growth, and near-tangency at  $k \rightarrow -1$ .

### E8 Solver

200 random seeds  $\times$  2 guess styles, periods 1–6, four maps.

### E9 Crossing correctness

The library’s crossings against an exact  $O(NM)$  brute force.

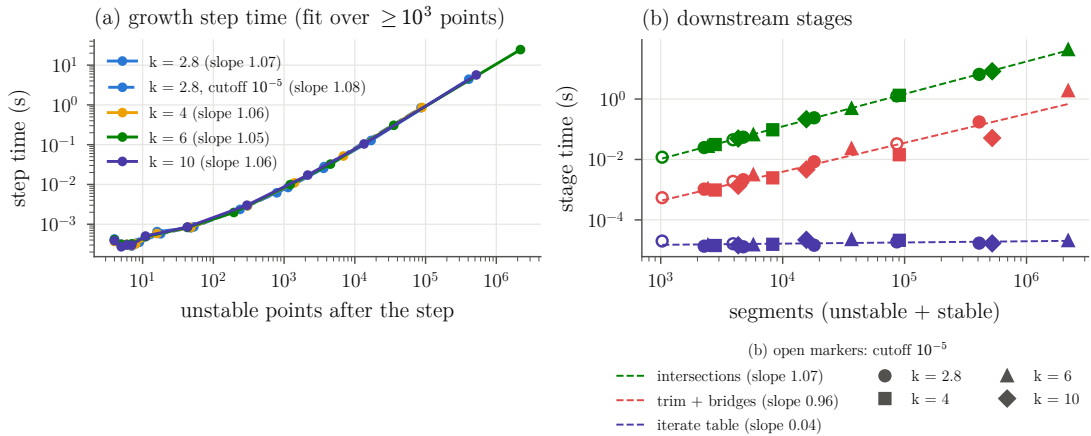
### E10 Determinism

Five fresh-process repeats on the CPU and five on the GPU.

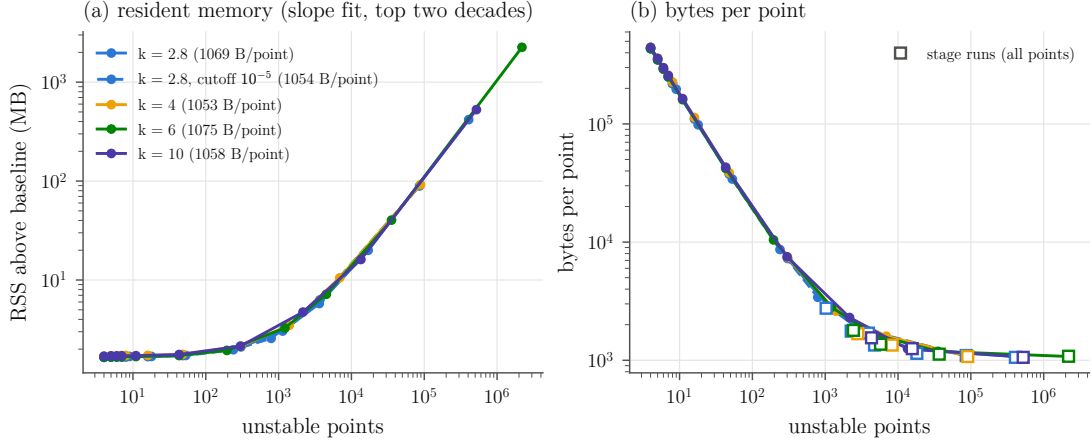
## 3 Performance and scaling (E3)

**Claim.** Growth, crossing detection and bridge construction all cost essentially linear time in the number of points, and memory costs about 1058 B per point. The binding constraint is not the algorithm but the number of points the escaping unstable arm generates.

**Evidence.** Figure 2(a) shows the time of one growth step against the points it produces, for four values of  $k$  and two cutoffs. Above  $10^3$  points the fitted slopes run from 1.05 to 1.08 (median 1.06). The downstream stages in Figure 2(b) scale with the segment count with slopes 1.07 (`compute_intersections`) and 0.958 (trim and bridges). The iterate table is flat (0.0391) because it depends on the crossings, not on the points. Resident memory (Figure 3) grows by 1058 B per point (median over runs; maximum 1075).

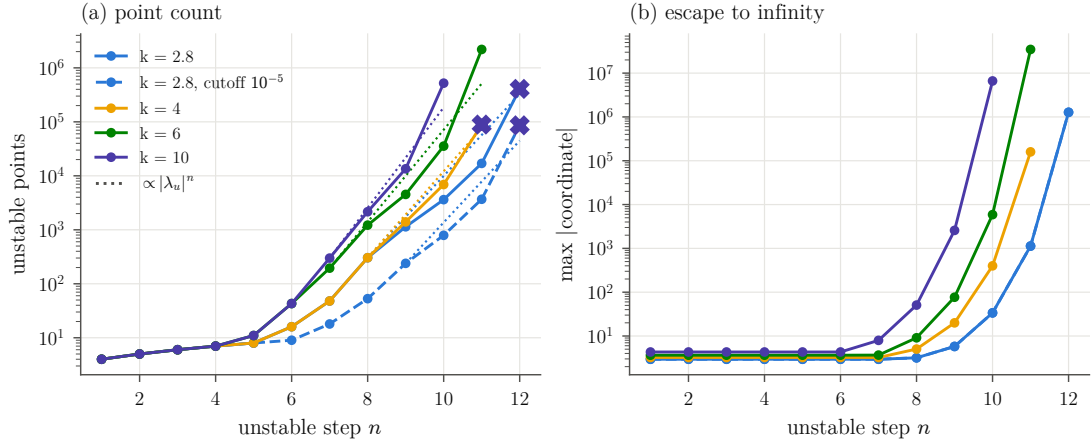


**Figure 2:** E3 cost. (a) Growth-step time against unstable points: all  $k$  and both cutoffs collapse onto one line with slope near 1 (legend), so cost depends on the point count alone. (b) Downstream stages against segments: intersections and trim/bridges are linear; the iterate table does not depend on the point count.



**Figure 3:** E3 memory. (a) RSS above the interpreter baseline: linear over the top two decades, with nearly the same slope for every  $k$  (legend). (b) Bytes per point: fixed interpreter overhead dominates small manifolds; the curve flattens onto the per-point slope for large ones.

Figure 4 explains why the point count is the real problem. For the first steps the points grow roughly like  $|\lambda_u|^n$ . Then, at step 9 ( $k = 10$ ) to 11 ( $k = 2.8$ ), the largest coordinate passes 100 and the growth factor departs from  $\lambda_u$ . On its last completed step the  $k = 6$  manifold grew by a factor 61.8 ( $\lambda_u = 7.15$ ),  $k = 10$  by 38.6 ( $\lambda_u = 8.52$ ) and  $k = 2.8$  by 24.0 ( $\lambda_u = 5.72$ ). Meanwhile the largest coordinate reached  $3.5 \times 10^7$  and  $6.6 \times 10^6$ .



**Figure 4:** E3 growth. (a) Unstable points per step against the  $|\lambda_u|^n$  guide (dotted). The curves follow it for a few steps and then shoot above it; crosses mark runs killed at the 24 GB cap. (b) The largest coordinate: the take-off in (a) coincides with the arm leaving for infinity.

**Interpretation.** The refinement places points to keep a fixed chord area, so the point count follows the arclength and curvature of the image, not the eigenvalue. Once the tip of the arm is at  $|x| \sim 10^3$ , one step of a quadratic map sends it to  $\sim 10^6$ , and the refinement faithfully resolves an arc that is irrelevant to the tangle. “More steps” therefore mostly buys points far outside the tangle. 3 growth runs were killed at the 24 GB cap. At that moment they held only 86 168–407 709 points after their last completed step, but they had spent 190–208 s inside the step that was killed.

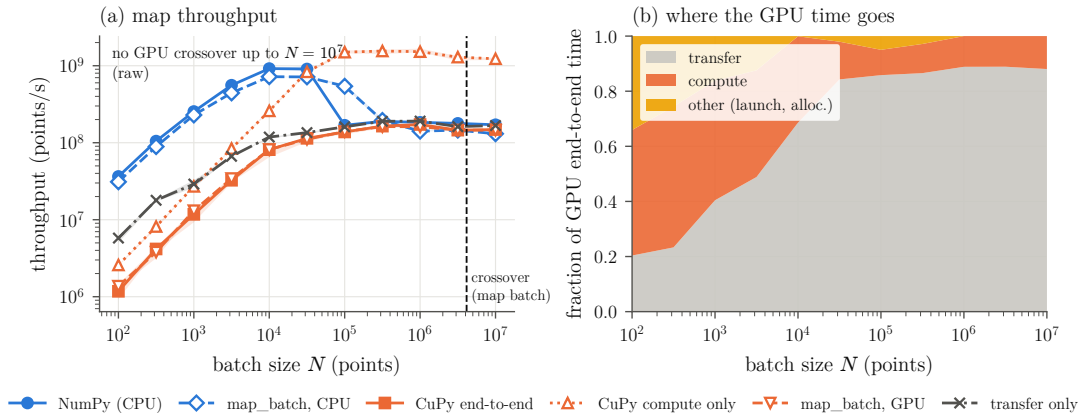
**Caveat.** The 1058 B/point slope gives a ceiling of roughly  $2.4 \times 10^7$  points in a 24 GB budget. This is an *estimate*: it extrapolates the steady-state slope, and a step’s transient working set comes on top of it. The exponents are fitted over  $\leq 3$  decades on one machine.

## 4 GPU acceleration (E1, E2)

**Claim.** Enabling the GPU (`enable_gpu`) does not speed up growth. Today it is neutral at best and slower at worst, and it cannot pay off while the map is a negligible share of the work.

### 4.1 Raw throughput

**Evidence.** Figure 5(a): NumPy peaks at  $9.2 \times 10^8$  points/s near  $N = 10^4$ , then drops by  $5.4\times$  to  $1.7 \times 10^8$  points/s. This is consistent with a cache effect: the working set leaves the per-core L2. CuPy on data already on the device reaches  $1.6 \times 10^9$  points/s, but the end-to-end path (host  $\rightarrow$  device  $\rightarrow$  host, which is what `gpu.py` does) peaks at  $1.7 \times 10^8$ . At  $N = 10^7$ , transfer is 88% of the GPU time (Figure 5(b)). The raw crossover: none up to  $N = 10^7$  (end-to-end CuPy reaches  $0.87\times$  of NumPy at the largest  $N$ ). Through `map_batch`, whose CPU path pays for transposes and copies, the GPU breaks even somewhere between  $6.2 \times 10^5$  (interpolated from the sweep) and  $4.1 \times 10^6$  (bisection). At  $10^7$  it is ahead by only  $1.11\times$ .



**Figure 5:** E1. (a) Throughput against batch size. Device-only compute (dotted triangles) is fastest, but every path that includes the transfer sits on the transfer-only line (grey crosses). The dashed line marks the bisected `map_batch` crossover; read it as the upper edge of a band, not a sharp threshold. (b) Share of GPU end-to-end time: transfer dominates from mid-size batches on.

### 4.2 End to end

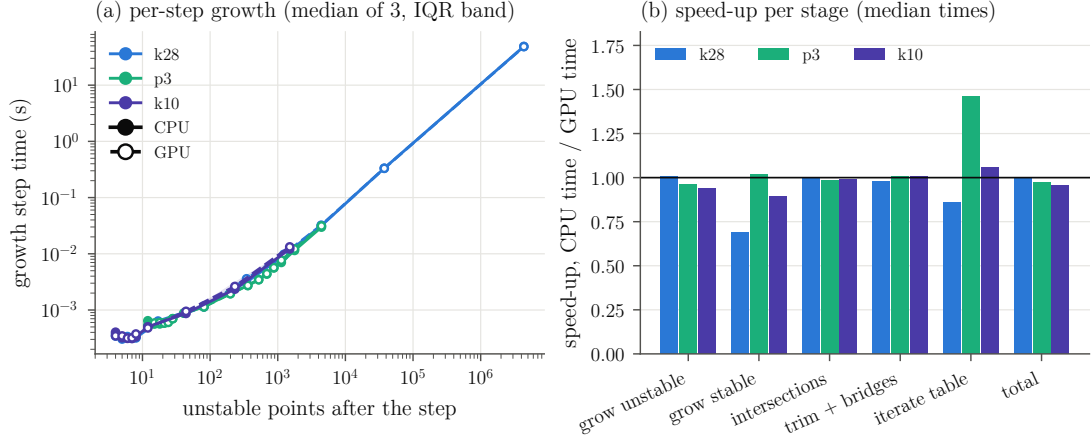
**Evidence.** Figure 6 and Table 2: the summed pipeline time on the GPU is  $0.999\times$  (k28),  $0.975\times$  (p3) and  $0.958\times$  (k10) of the CPU speed (median of three repeats; best repeats  $1.004\times$ ,  $0.991\times$ ,  $1.002\times$ ). The best single growth step, k28 at  $4.3 \times 10^6$  points, gains  $1.007\times$ . On the E10 case the GPU is clearly slower:  $0.85\times$  (Section 11).

**Table 2:** E2: median CPU time / median GPU time per stage, the CPU map share  $f_{\text{map}}$  (cProfile) and the Amdahl bound  $1/(1 - f_{\text{map}})$ .

| case | grow unstable | grow stable   | intersections | trim + bridges | iterate table | total         | $f_{\text{map}}$ | Amdahl        |
|------|---------------|---------------|---------------|----------------|---------------|---------------|------------------|---------------|
| k28  | $1.007\times$ | $0.689\times$ | $1.001\times$ | $0.978\times$  | $0.858\times$ | $1.001\times$ | 0.0171%          | $1.000\times$ |
| p3   | $0.962\times$ | $1.017\times$ | $0.985\times$ | $1.006\times$  | $1.462\times$ | $0.970\times$ | 0.505%           | $1.005\times$ |
| k10  | $0.939\times$ | $0.895\times$ | $0.991\times$ | $1.006\times$  | $1.060\times$ | $0.958\times$ | 0.84%            | $1.008\times$ |

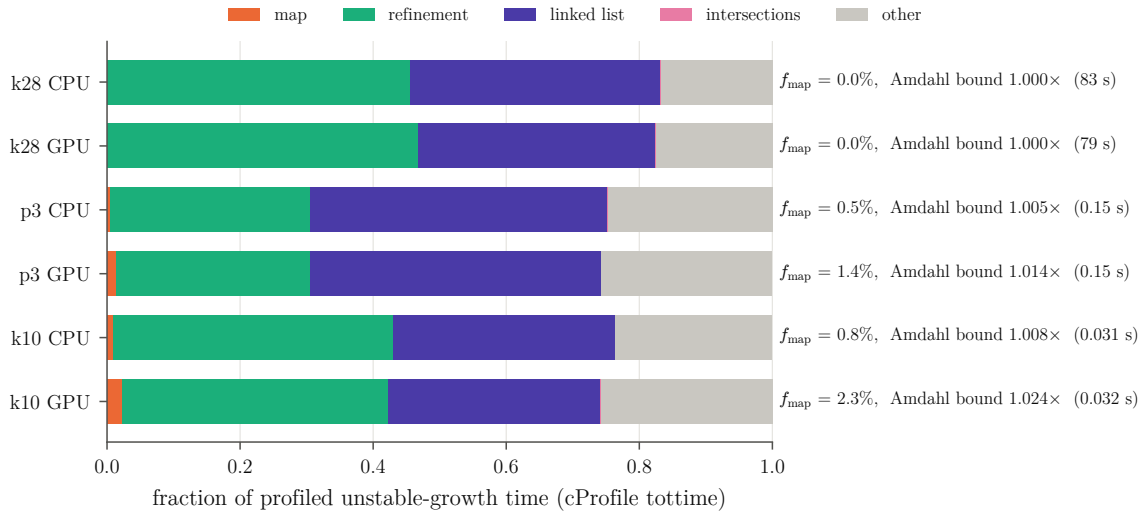
### 4.3 Why: Amdahl's law

**Evidence.** Figure 7: in the cProfile of unstable growth the map is 0.0171% (k28), 0.505% (p3) and 0.84% (k10) of the CPU time. The bounds are  $1.000\times$ ,  $1.005\times$  and  $1.008\times$ . Refinement plus



**Figure 6:** E2. (a) Per-step growth time; CPU (filled) and GPU (open) markers lie on top of each other up to  $4.3 \times 10^6$  points. (b) Per-stage speed-up. Everything sits within a few percent of 1 except tiny stages, where sub-millisecond noise dominates (p3 iterate table, k28 stable growth).

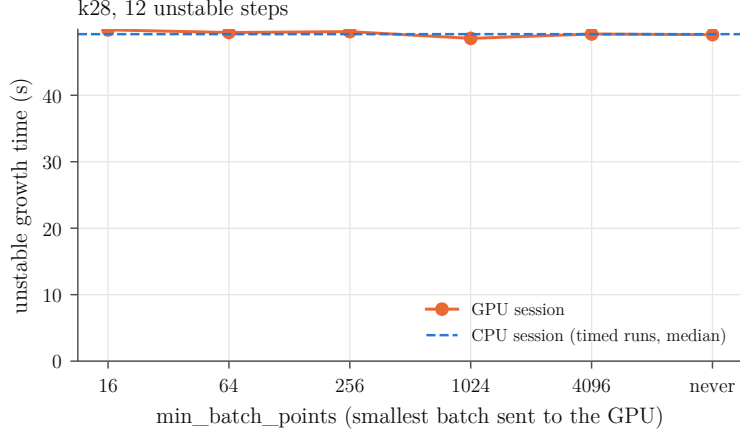
linked-list work is 83% (k28), 75% (p3) and 76% (k10). In the k28 profile (83 s), `_refine_layer` alone takes 44% of the self time. `Point.__init__` runs  $8.7 \times 10^6$  times (18% cumulative), each paired with a `numpy.array` call ( $8.7 \times 10^6$ ). The `min_batch_points` sweep (Figure 8) is flat: the best setting (1024) is  $1.011\times$  faster than “never”, which is inside the run-to-run noise of Section 11.



**Figure 7:** E2 cProfile breakdown of unstable growth (self time by category). The map (orange) is invisible at this scale; refinement and the linked list (Python objects, pointer walks) take 75%–83% of it. The Amdahl bound on the right is the most any map accelerator could give.

**Interpretation.** The GPU path accelerates the only part of growth that was already fast. Growth is a Python object graph: one `Point` per vertex, two doubly linked lists, and refinement that walks them layer by layer. For the GPU to pay, refinement and the linked list must become arrays (coordinates, cdists, iterate indices), with the whole growth loop resident on the device. Only then would  $f_{\text{map}}$  become the GPU-friendly bulk of the work. Short of that, the measured ceiling is  $1.008\times$ .

**Caveat.** cProfile inflates the per-call overhead of Python functions relative to NumPy kernels, so the true  $f_{\text{map}}$  is probably a little larger than measured. Even ten times the measured share would keep the bound close to 1. Three repeats per configuration resolve differences of a few percent, not less.

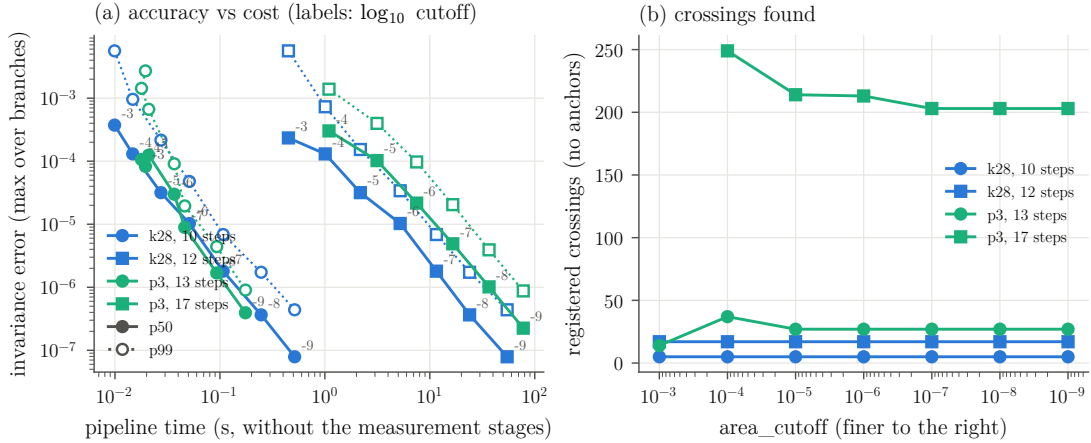


**Figure 8:** E2 `min_batch_points` sweep on k28 (12 unstable steps). The GPU session is flat across the threshold and matches the CPU median (dashed): the threshold has nothing to tune.

## 5 Accuracy versus cost (E4)

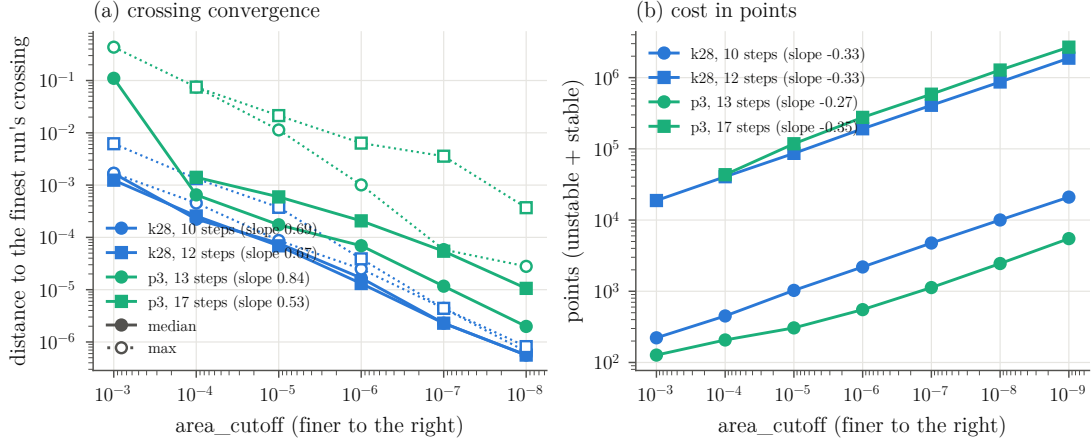
**Claim.** `area_cutoff` trades accuracy for points along a predictable power law. The default  $10^{-4}$  is adequate for the period-1 cases but not for p3.

**Evidence.** Figure 9(a) plots invariance error against pipeline time. The invariance error is the distance from the image of a point inside a polyline segment to the polyline of the image branch; the probe samples segment interiors because a vertex’s image is, by construction, often a vertex. On k28 (10 steps), going from  $10^{-4}$  to  $10^{-7}$  moves the median error from  $1.3 \times 10^{-4}$  to  $1.8 \times 10^{-6}$  and the 99th percentile from  $9.6 \times 10^{-4}$  to  $6.9 \times 10^{-6}$ . The cost goes from 0.0148s to 0.107s and from 450 to 4757 points. Over cutoffs  $10^{-6}$ – $10^{-9}$  the median error has slopes 0.70 and 0.70 (k28, 10 and 12 steps), and 0.64 and 0.66 (p3, 13 and 17 steps). The point count has slope -0.332 (k28) and -0.272 (p3) (Figure 10(b)). Crossing positions converge to the finest run with slopes 0.73, 0.70, 0.66 and 0.58 (Figure 10(a)). The full grid is in Table 8.



**Figure 9:** E4. (a) Accuracy against cost; labels are  $\log_{10}$  of the cutoff. Every case traces a straight Pareto front, so there is no knee and no cheap win. (b) Registered crossings per cutoff. k28 is stable at every cutoff; p3 needs  $\leq 10^{-5}$  (13 steps) or  $\leq 10^{-7}$  (17 steps) before the count settles.

**Interpretation.** The exponents are what chord geometry predicts. A chord of length  $\ell$  on a curve of curvature  $\kappa$  deviates by  $h \approx \kappa \ell^2 / 8$  and encloses an area  $\propto \kappa \ell^3$ . A fixed area cutoff  $A$  therefore gives  $\ell \propto A^{1/3}$ , so points  $\propto A^{-1/3}$  and a geometric error  $h \propto A^{2/3}$ . The crossing *count* is what matters for topology. For p3 at 13 steps the default  $10^{-4}$  registers 37 crossings and  $10^{-3}$  registers 14, while every cutoff from  $10^{-5}$  on gives 27. At 17 steps,  $10^{-4}$  gives 249 and  $10^{-5}$  gives 214,



**Figure 10:** E4 convergence. (a) Distance of each finest-run crossing to the nearest crossing of the coarser run: steady power-law convergence; p3 at 17 steps is the slowest. (b) Cost in points: slopes -0.332 and -0.272, close to the predicted  $-1/3$ .

against 203 from  $10^{-7}$  on. A coarse polygon both misses true crossings and creates spurious ones.  $10^{-7}$  is the cheapest setting that is converged in every case tested.

**Caveat.** The reference is the finest run ( $10^{-9}$ ), not an exact manifold, so the convergence plots measure self-consistency. The invariance error is a reference-free proxy, but it measures one-step consistency, not distance to the true manifold. p3 at 17 steps and  $10^{-3}$  ran out of memory (16 GB cap): a coarse polygon of the escaping arm is not cheaper.

## 6 Physical invariants (E5)

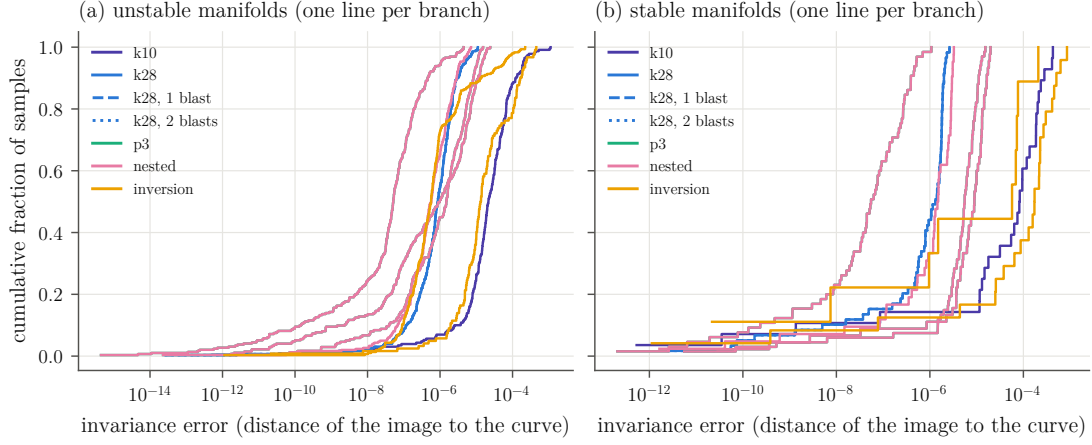
**Claim.** On every frozen case the numerics respect what the dynamics requires: fixed points to machine precision, monotone cdists, manifolds mapped onto themselves to within the refinement tolerance, and lobe areas preserved.

**Evidence.** Table 3 summarises the cases. The fixed-point residual is at most  $8.9 \times 10^{-16}$  and the eigenvector residual at most  $1.8 \times 10^{-15}$ . Along the manifolds, 77 328 cdist steps (and 317 502 along bridges) contain 0 decreases and 0 ties. No sampled image lands beyond the tail of its image branch (0 such images). The largest invariance error,  $1.1 \times 10^{-3}$ , belongs to k10, one of the two cases (with inversion) grown at the default  $10^{-4}$  (Figure 11). The 70 registered iterate links agree with the map to a median  $6.4 \times 10^{-6}$  (maximum  $1.5 \times 10^{-3}$ , again k10), so the error tracks each case's cutoff (Figure 12(a)). The 57 lobe/image-lobe pairs have  $|A'/A - 1| \leq 5.0 \times 10^{-3}$  with 0 sign flips (Figure 12(b)). The k28 lobes measure  $\pm 7.004$ , and their images agree to  $5.3 \times 10^{-6}$  in absolute terms. The p3 lobes are tiny (median  $1.1 \times 10^{-4}$ ), so their larger *relative* error corresponds to an absolute error of at most  $5.4 \times 10^{-7}$ , a few times the cutoff.

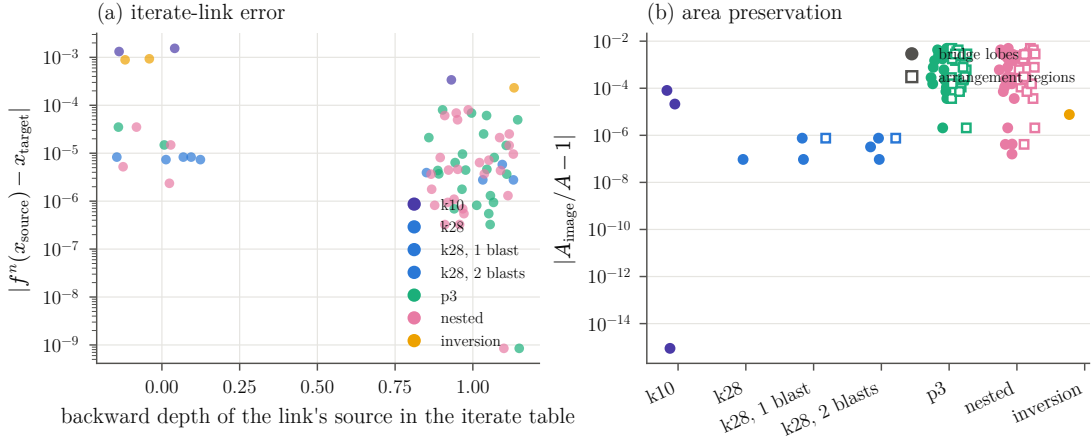
**Table 3:** E5 per-case invariants (maxima over orbit points and branches). “arr. image” counts the arrangement regions whose `image_of` was found/missing.

| case           | outcome | $ f^p(x^*) - x^* $    | $  \lambda_u \lambda_s  - 1 $ | eigvec res.           | inv. max             | lobe $ A'/A - 1 $    | cdist dec./ties (man.) | cdist dec./ties (bridges) | arr. image found/missing |
|----------------|---------|-----------------------|-------------------------------|-----------------------|----------------------|----------------------|------------------------|---------------------------|--------------------------|
| k10            | ok      | 0                     | $1.2 \times 10^{-11}$         | $1.8 \times 10^{-15}$ | $1.1 \times 10^{-3}$ | $8.1 \times 10^{-5}$ | 0/0                    | 0/0                       | 0/7                      |
| k28 unblasted  | ok      | $8.9 \times 10^{-16}$ | 0                             | $5.6 \times 10^{-17}$ | $1.1 \times 10^{-5}$ | $9.4 \times 10^{-8}$ | 0/0                    | 0/0                       | 0/3                      |
| k28 one blast  | ok      | $8.9 \times 10^{-16}$ | 0                             | $5.6 \times 10^{-17}$ | $1.1 \times 10^{-5}$ | $7.6 \times 10^{-7}$ | 0/0                    | 0/0                       | 1/4                      |
| k28 two blasts | ok      | $8.9 \times 10^{-16}$ | 0                             | $5.6 \times 10^{-17}$ | $1.1 \times 10^{-5}$ | $7.6 \times 10^{-7}$ | 0/0                    | 0/0                       | 1/8                      |
| p3             | ok      | 0                     | $4.4 \times 10^{-16}$         | $2.2 \times 10^{-16}$ | $2.4 \times 10^{-5}$ | $5.0 \times 10^{-3}$ | 0/0                    | 0/0                       | 22/3                     |
| nested         | ok      | $4.4 \times 10^{-16}$ | $4.4 \times 10^{-16}$         | $2.3 \times 10^{-16}$ | $2.4 \times 10^{-5}$ | $5.0 \times 10^{-3}$ | 0/0                    | 0/0                       | 23/10                    |
| inversion      | ok      | 0                     | 0                             | $1.4 \times 10^{-16}$ | $8.6 \times 10^{-4}$ | $7.6 \times 10^{-6}$ | 0/0                    | 0/0                       | 0/4                      |

**Interpretation.** The area check is the strongest of these results. It compares polygons built from independently refined arcs, so a systematic drift would show up as a ratio that grows with depth, and none does.



**Figure 11:** E5 invariance error, one cumulative distribution per branch. The curves to the right are k10 and the inversion saddle, the two cases grown at the default cutoff  $10^{-4}$ ; the  $10^{-7}$  cases sit well to the left.



**Figure 12:** E5. (a) Error of each registered iterate link: the links of the default-cutoff cases (k10, inversion) sit highest, those of the  $10^{-7}$  cases lower. (b) Relative area change of each lobe under one map step (filled) and of each arrangement region with a found image (open). All are  $\leq 5.0 \times 10^{-3}$ ; the largest values belong to the tiny p3 lobes. (The legend overlaps a few points; it hides no outlier.)

**Caveat.** (i)  $||\lambda_u \lambda_s| - 1|$  is *weak* evidence: the eigenvalues come from `np.linalg.eig` of the analytic Jacobian, whose determinant is exactly 1, so the defect is  $4.4 \times 10^{-16}$  by construction. The exception confirms the point: k10 passes no analytic Jacobian, its stored Jacobian deviates from the analytic one by  $1.4 \times 10^{-12}$ , and its defect is  $1.2 \times 10^{-11}$ . (ii) The arrangement’s `image_of` found an image for 47 regions and none for 39. It found them for most of p3 and nested, for one region of each blasted k28 case, and for none of k10, unblasted k28 or inversion (Table 3). The data does not say whether this is a gap in `image_of` or an expected consequence of image crossings that are not registered. I list it as an open question (Section 13), not as a bug.

## 7 Correctness of crossing detection (E9)

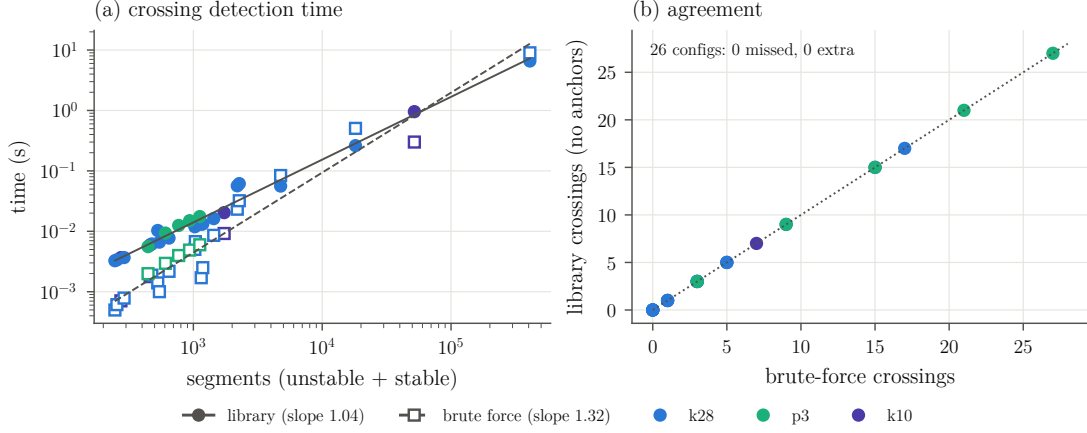
**Claim.** For transverse crossings the R-tree path finds exactly the crossings of the polygons, no more and no less.

**Evidence.** Over 26 configurations (Table 4, Figure 13), an exact orientation-test brute force found 154 proper  $u \times s$  crossings. The registry matched 154, with 0 missed (none needed the near-parallel, zigzag or collision explanations) and 0 extra.

**Interpretation.** Speed is not where the R-tree path shines at these sizes: its slope is 1.04 against

**Table 4:** E9: crossings against brute force, summed per case. \*Some configurations were too large for the  $u \times u$  check.

| case | configs | brute | matched | near-par. | zigzag | collision | missed unexpl. | extra touch | extra unexpl. | $u \times u$ |
|------|---------|-------|---------|-----------|--------|-----------|----------------|-------------|---------------|--------------|
| k10  | 4       | 26    | 26      | 0         | 0      | 0         | 0              | 0           | 0             | 0*           |
| k28  | 17      | 53    | 53      | 0         | 0      | 0         | 0              | 0           | 0             | 4*           |
| p3   | 5       | 75    | 75      | 0         | 0      | 0         | 0              | 0           | 0             | 0            |



**Figure 13:** E9. (a) Detection time against segments. The library is linear; the vectorised brute force is steeper but faster on all but the largest configurations. (b) Library against brute-force counts: every point lies on the diagonal.

1.32 for the brute force, but the brute force was faster in 23 of 26 configurations. On the largest ( $4.1 \times 10^5$  segments) the library took 6.6s and the brute force 9.0s (the largest brute/library ratio was  $1.932\times$ ). This is not a problem: the stable side is short, so the brute force is effectively linear here, and the library time includes registration and bookkeeping. The asymptotic advantage remains with the library.

**Caveat.** Two gaps. First, the polygons *can* cross themselves: 4 proper  $u \times u$  crossings appeared in 2 configurations, all at cutoff  $10^{-5}$ , and 2 large configurations were not checked. A manifold cannot cross itself (CLAUDE.md), so these are discretisation artefacts. The library never tests  $u \times u$  pairs, so nobody is told. A cheap self-crossing check would make a good resolution diagnostic. Second, an API trap: `compute_intersections(reset=True)` *replaces* the `IntersectionRegistry` object, so a caller that cached `workbench.intersection_registry` beforehand holds a stale, empty registry. This is documented in the docstring, but easy to trip over (the E6 script carries a comment about it).

## 8 Robustness across parameters (E6)

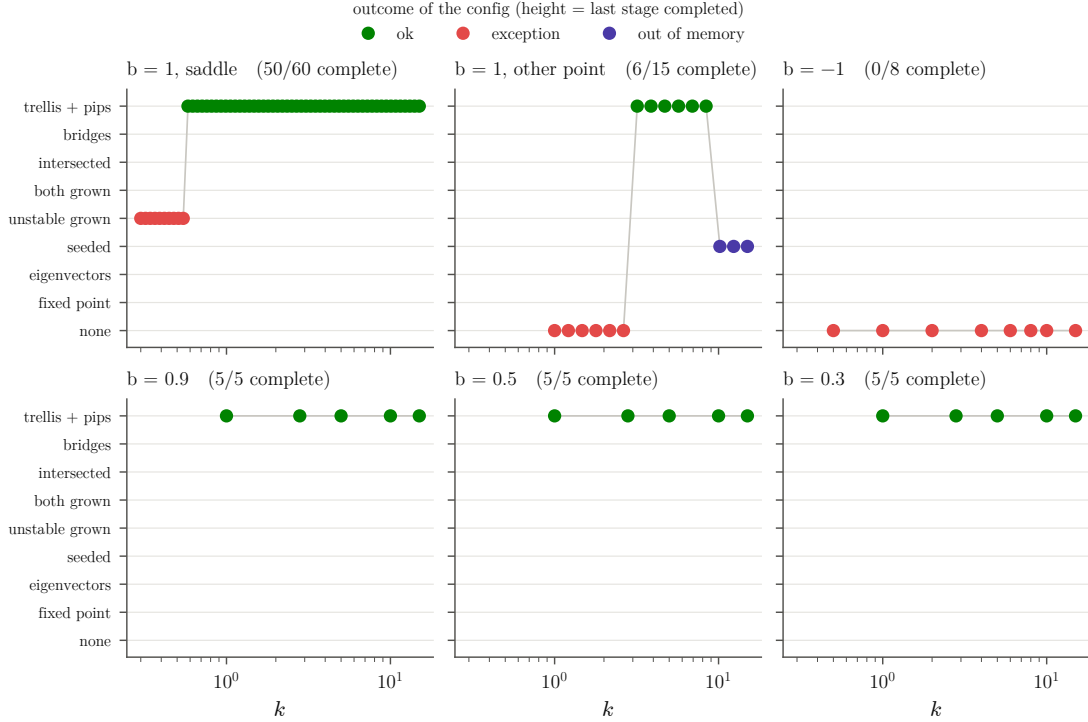
**Claim.** One fixed recipe runs end to end over most of the area-preserving  $b = 1$  family. It fails cleanly where it should, but at small  $k$  it produces tangles that are numerically fine and topologically empty.

**Evidence.** Figure 14 and Table 5: 71 of 98 configurations (72.4%) reached the trellis, and 83.3% of the  $b = 1$  saddles did. The failure classes are:

- **Small  $k$ :** 10 saddles with  $k \in [0.30, 0.545]$  grew the unstable side and then raised “Max iterations reached” in `grow_until_turnaround` on the stable side. The smallest completed  $k$  is 0.582.
- **Elliptic point:** 6 configurations ( $k \leq 2.63$ ) were rejected as “not a saddle”, by design.
- **Inversion saddle:** 3 runs with  $k \geq 10.2$  hit the 4 GB cap during growth, the same blow-up as

in Section 9.2.

- **Orientation reversing  $b = -1$ :** all 8 were rejected (“eigenvalues of disagreeing sign”), by design.
- **Dissipative  $b$ :** 15/15 completed. Nothing in the pipeline checks area preservation, so completion here says nothing about correctness.



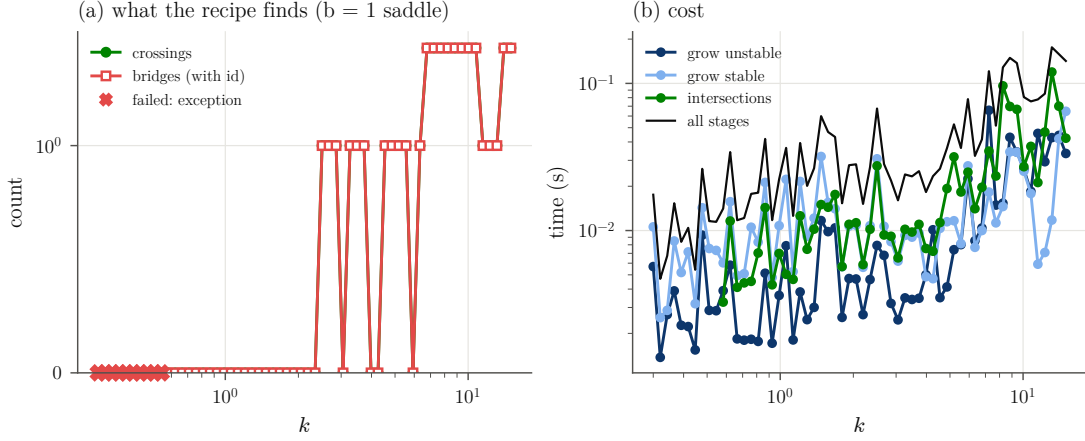
**Figure 14:** E6. Last stage completed per configuration (height) and outcome (colour). Read the  $b = 1$  saddle panel left to right: a block of small- $k$  exceptions, then complete runs. The other point fails as elliptic below  $k = 3$  and runs out of memory as the inversion saddle at large  $k$ .

**Table 5:** E6: configurations per family by last stage completed.

| family                | none | fixed point | eigen | seeded | grown unstable | grown intersected | bridges | trellis pips | total |
|-----------------------|------|-------------|-------|--------|----------------|-------------------|---------|--------------|-------|
| $b = -1$              | 8    | 0           | 0     | 0      | 0              | 0                 | 0       | 0            | 8     |
| $b = 0.3$             | 0    | 0           | 0     | 0      | 0              | 0                 | 0       | 5            | 5     |
| $b = 0.5$             | 0    | 0           | 0     | 0      | 0              | 0                 | 0       | 5            | 5     |
| $b = 0.9$             | 0    | 0           | 0     | 0      | 0              | 0                 | 0       | 5            | 5     |
| $b = 1$ , other point | 6    | 0           | 0     | 3      | 0              | 0                 | 0       | 6            | 15    |
| $b = 1$ , saddle      | 0    | 0           | 0     | 0      | 10             | 0                 | 0       | 50           | 60    |

**Interpretation.** “Complete” is not “useful”. Of the 50 completed  $b = 1$  saddles, 26 have *no* crossing at all (up to  $k = 5.93$ ). The first crossing appears at  $k = 2.50$ , and no configuration has more than 3 (Figure 15). With a fixed 8-step recipe the manifolds at small  $k$  are simply too short to meet, because  $\lambda_u$  is small. A recipe has to scale its depth with  $1/\ln \lambda_u$ , as E7’s tangency study does.

**Caveat.** The “Max iterations” failures are a limit of the stable-side stop condition with this recipe, not necessarily of the map. A larger cap or a different stop condition might rescue them; I did not try either. Note also that the 4 GB cap of E6 is lower than elsewhere.



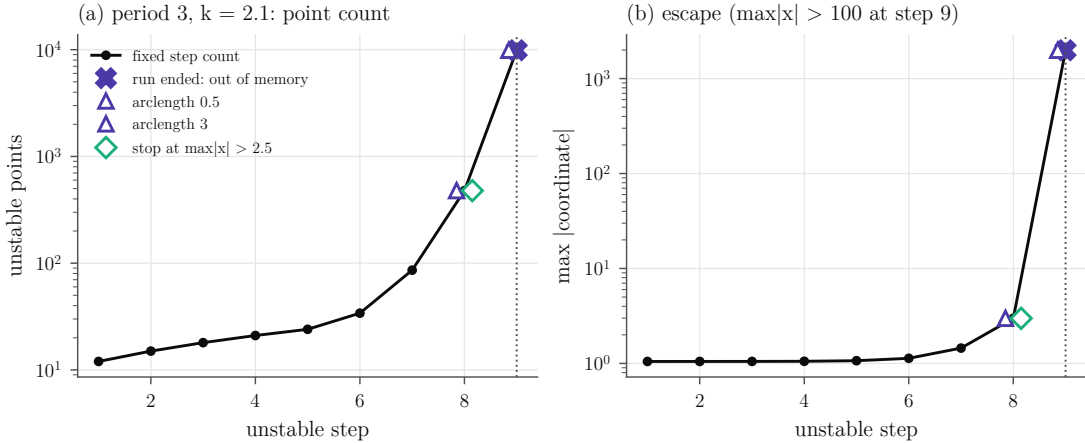
**Figure 15:** E6  $b = 1$  saddle. (a) Crossings and bridges found by the fixed recipe: none below  $k = 2.50$  and intermittent up to  $k = 5.93$ . (b) Stage times across  $k$ : cheap everywhere; cost is not what limits this sweep.

## 9 Failure modes and breaking points (E7)

### 9.1 Escape to infinity

**Claim.** The documented escape reproduces, and the documented mitigation works.

**Evidence.** Figure 16: on the  $k = 2.1$  period-3 orbit with a fixed step count, the run completed 9 steps holding 9881 points, with  $\max|x| = 2.0 \times 10^3$ . It was then killed at the 20 GB cap (peak 20 496 MB) after 175s inside the next step. The same arm grown with a stop condition (arclength 0.5, or  $\max|x| > 2.5$ ) stopped safely at 478 points; arclength 3 stopped at 9881.



**Figure 16:** E7 escape ( $k = 2.1$ , period 3). (a) Points per step: the fixed count dies one step after the curve turns up; the stop conditions (open markers) end before the take-off. (b) The largest coordinate jumps in the last completed step, which is what the  $\max|x|$  stop condition catches.

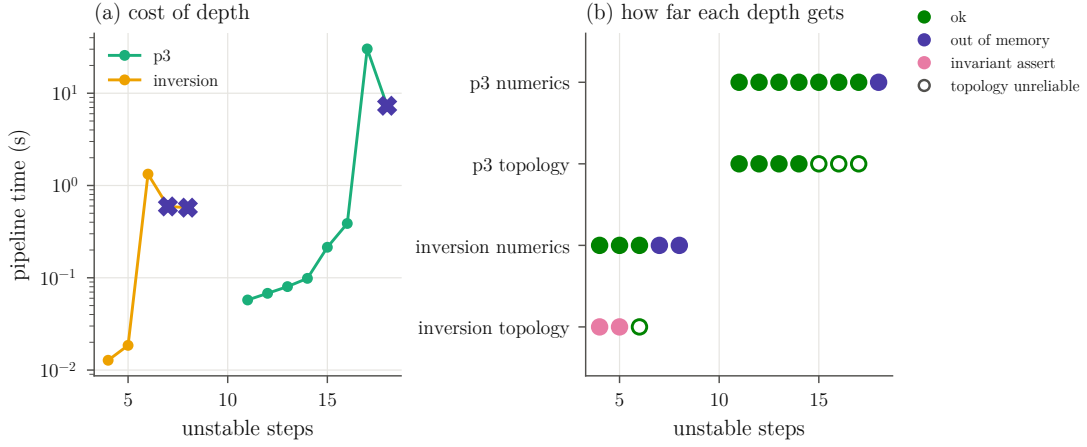
**Interpretation.** `grow_n_times` has no notion of budget. A single step can go from harmless to tens of gigabytes, so stop-condition growth should be the recommended interface.

### 9.2 Depth

**Claim.** p3 is numerically robust up to 17 unstable steps, but its topology stops being reliable earlier. The inversion saddle is fragile at both ends.

**Evidence.** Figure 17: p3 completes the numerics through 17 steps (584 747 points); 18 steps hit the 20 GB cap. Its topology reports `is_reliable` up to 14 steps and `is_reliable=False` from 15

on. The inversion saddle completes the numerics only up to 6 steps. At 7 and 8 steps it is killed at the cap after reaching 49 417 points at step 6. So the documented “7 steps does not finish” is a memory blow-up, not a hang. At 4 and 5 steps the numerics succeed, but the topology stack trips its own invariant: “crossing 0 at stable cdist 0.0 ... is owned by 0 partition elements”.

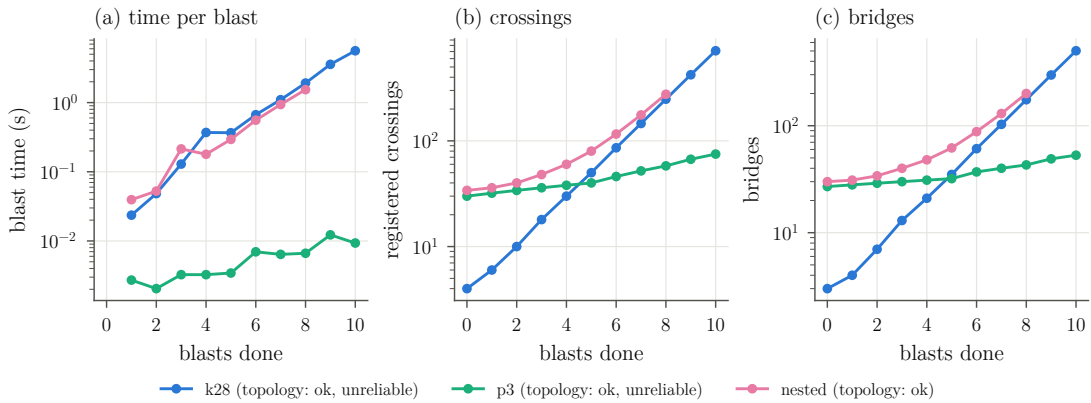


**Figure 17:** E7 depth. (a) Pipeline time against unstable steps; p3 climbs steeply past 15 steps. (b) Outcome by layer. The numerics fail only by memory; open circles are topology runs that finished with `is_reliable=False`.

**Caveat.** The inversion assertion lives in the topology layer and is recorded here because the stress test reached it. Whether the shallow inversion trellis is too thin to partition or the assertion is too strict is for the topology owner to decide.

### 9.3 Blasts

**Evidence.** Figure 18: k28 and p3 survive 10 and 10 blasts and nested survives 8, all without numerical failure. k28’s crossings grow from 4 to 712, and its last blast took 5.6s. Afterwards the topology reports k28 as ok (unreliable) (classes: 32, unresolved: 7, virtual: 2) and p3 as ok (unreliable) (classes: 31, unresolved: 1, virtual: 1). Nested stays ok (classes: 13, none unresolved or virtual).



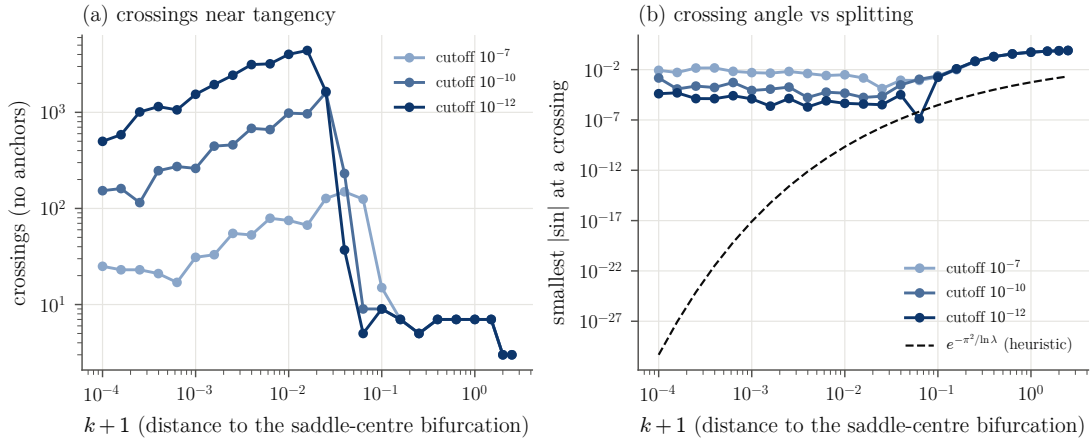
**Figure 18:** E7 blasts. (a) Time per blast grows geometrically for k28 and nested and stays at milliseconds for p3. (b,c) Crossings and bridges grow geometrically. Nothing breaks numerically within 8–10 blasts.

**Interpretation.** Blasting is numerically sound to the depth tested. The limit is downstream: past a few blasts the topology cannot classify everything it is given.

## 9.4 Near tangency: artefact crossings

**Claim.** This is the most important limitation found. As  $k \rightarrow -1$  (the saddle-centre bifurcation) the true separatrix splitting becomes exponentially small. The numerics then register polygon-on-polygon crossings as if they were transverse, their number is set by the resolution, and no warning fires.

**Evidence.** Figure 19: 72 configurations, with  $k+1$  log-spaced from  $10^{-4}$  to 2.5 and three cutoffs. At  $k+1 = 10^{-4}$  the registered crossings are 25, 153 and 499 for cutoffs  $10^{-7}$ ,  $10^{-10}$  and  $10^{-12}$ . The peaks over  $k$  are 149, 1607 and 4401. From  $k+1 \geq 0.16$  on, all three cutoffs agree (e.g. 7 crossings at  $k+1 = 1$ ). At the finest cutoff and  $k+1 \leq 0.025$ , the smallest  $|\sin|$  per configuration lies between  $2.0 \times 10^{-6}$  and  $4.9 \times 10^{-5}$  (median of the medians  $4.8 \times 10^{-4}$ ). The heuristic splitting  $e^{-\pi^2/\ln \lambda}$  over the same range is  $4.6 \times 10^{-31}$  to  $1.9 \times 10^{-8}$ . Across all of this, warnings of any kind: 0. The smallest angle anywhere,  $1.3 \times 10^{-7}$ , sits at the transition, not near  $k = -1$ .



**Figure 19:** E7 tangency. (a) Crossings against  $k+1$ : below  $k+1 \approx 0.16$  the three cutoffs disagree strongly. That is the signature of artefacts, since a true crossing cannot depend on the polygon’s resolution. (b) Smallest crossing angle against the heuristic splitting (dashed): the registered angles are many decades larger than the true splitting, i.e. they measure polygon noise.

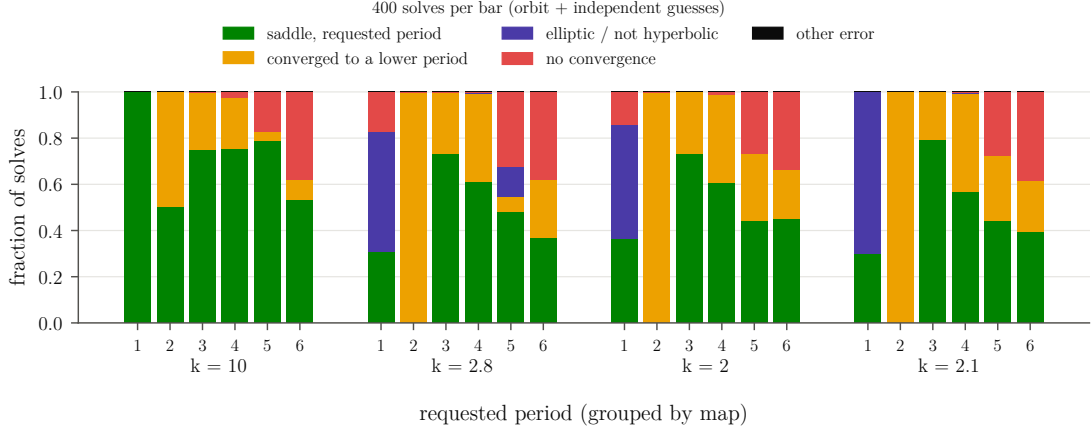
**Interpretation.** When the true separation of the manifolds is far below the polygon’s own geometric error (Section 5), the two polygons interleave at random. Each interleaving is a clean, well-conditioned segment crossing (median  $|\sin|$   $4.8 \times 10^{-4}$ ), far from the library’s relative near-parallel threshold of  $10^{-12}$ , so it is registered as transverse. The crossing tests are correct about the *polygons*; the polygons are not accurate enough to resolve the *manifolds*.

**Caveat.** The splitting reference is a heuristic order of magnitude, not a computed splitting. Some of the near-tangent crossings may be real; the claim is that their *count* is not trustworthy. A guard that compares each crossing’s angle and position with the local invariance error would flag this regime.

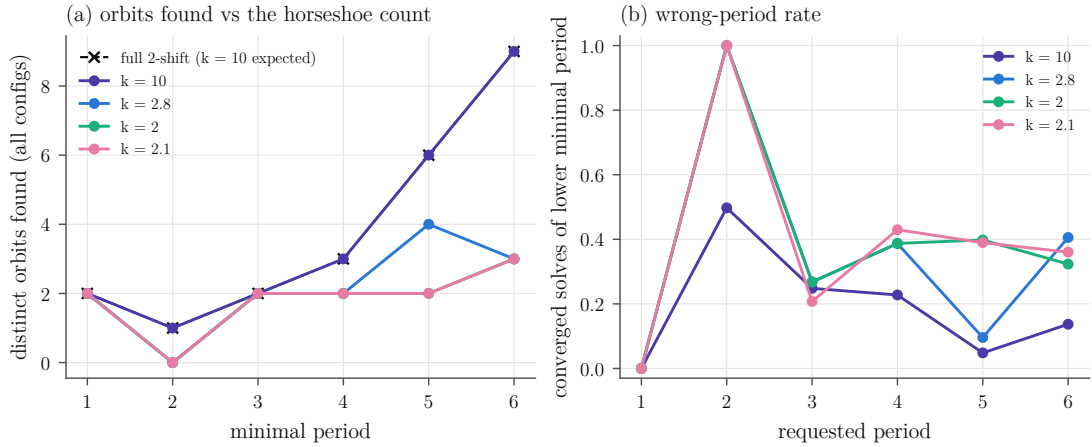
## 10 Fixed-point solver (E8)

**Claim.** The solver is fast and accurate when it finds the orbit you asked for. It finds the full set of short orbits of the  $k = 10$  horseshoe. It does *not* check the minimal period, so it regularly hands back a lower-period orbit as if it were period  $p$ .

**Evidence.** 9600 solves (Figure 20, Table 9 in the appendix): 88% converged and 12% did not. 49.7% of all solves gave a saddle of the requested minimal period. The maximum orbit residual is  $5.8 \times 10^{-11}$ , and the median solve takes 0.67 ms (99th percentile 7.2 ms, Figure 22). At  $k = 10$  the distinct orbits found equal the full 2-shift count for 6/6 periods (23 of 23 orbits, Figure 21(a)).



**Figure 20:** E8 outcomes per map and requested period. Orange (“converged to a lower period”) is the trap; the all-orange period-2 bars at  $k < 3$  are maps with no period-2 orbit at all.



**Figure 21:** E8. (a) Distinct orbits found: at  $k = 10$  the curve lies exactly on the 2-shift count. (b) Fraction of converged solves with a lower minimal period, per requested period.

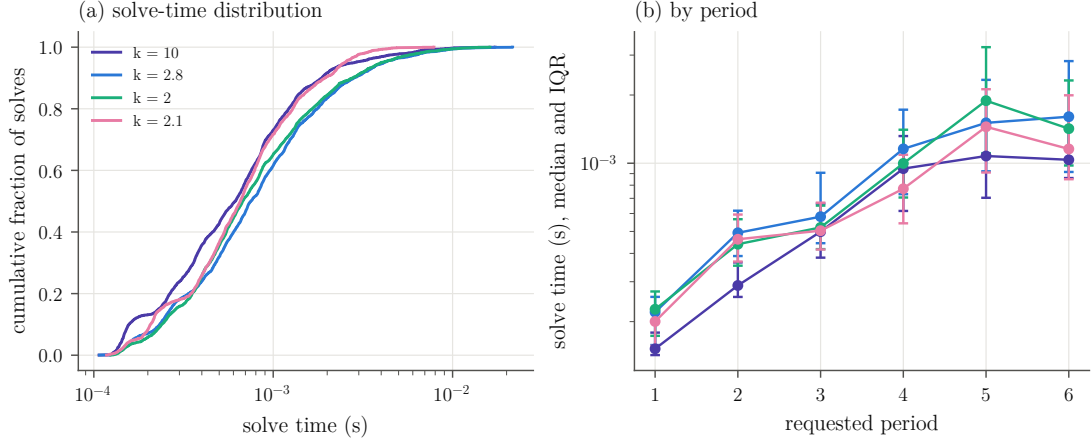
**Interpretation.** The headline wrong-period rate, 34.8% of converged solves, mixes two different things. 1198 solves (14.2%) asked for period 2 at  $k < 3$ . There is no period-2 orbit there: none was found from any seed, consistent with its birth in the period doubling of the elliptic point at  $k = 3$ . So falling back to a fixed point is the only thing a shooting solver can do. The other 1744 solves (20.6%) are genuine slides: the requested period exists, and the solver found a fixed point or a divisor-period orbit instead. At  $k = 10$ , where every period exists, the rate is 20.1%. The real problem is downstream: 1424 lower-period solves passed `construct_fixed_point`, which built a saddle with the *requested* period (e.g. a period-1 point labelled period 2 with  $k_{\text{value}} = 2$ ). The lower-period solves that were rejected were rejected for being elliptic, not for their period.

**Caveat.** Seeds are uniform in a box sized to each map. Basins elsewhere are not sampled, so the orbit counts at  $k < 3$  are lower bounds.

## 11 Reproducibility (E10)

**Claim.** Runs are deterministic on a given device, and CPU and GPU agree to floating-point rounding.

**Evidence.** Table 6 and Figure 23: across 5 fresh-process repeats per device, CPU–CPU is identical and GPU–GPU is identical (sha256 of every point array). CPU–GPU differs in 5 of 5 hashes, and still differs in 3 of 5 hashes after rounding to  $10^{-12}$ . The crossing coordinates agree to  $2.2 \times 10^{-16}$ .

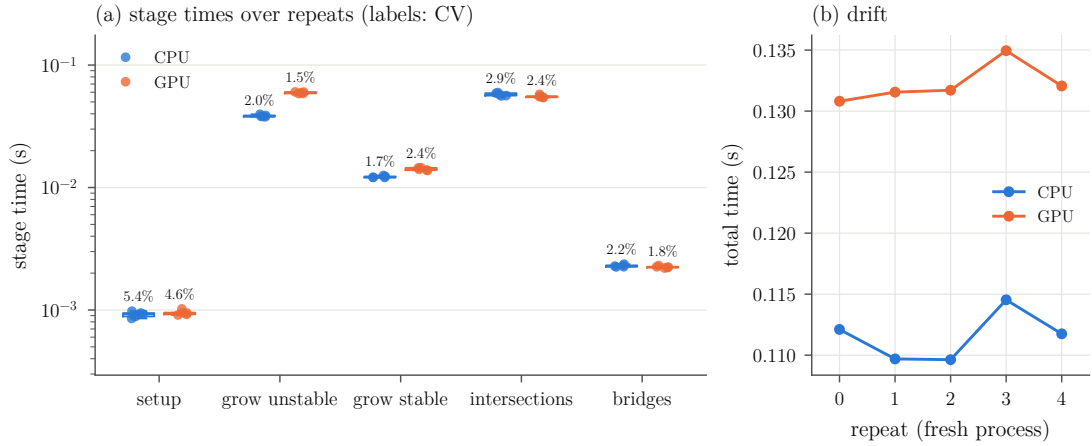


**Figure 22:** E8 solve time. (a) Distribution: sub-millisecond for most solves. (b) Median and IQR by period: cost grows mildly with the period.

The CV of the summed stage time is 1.82% (CPU) and 1.21% (GPU); per stage it is at most 5.41% and 4.56%. In this case the GPU run is slower, at  $0.85\times$  of the CPU speed (0.112s against 0.132s).

**Table 6:** E10: do the sha256 hashes agree across repeats (exact bytes, and rounded to  $10^{-12}$ )?

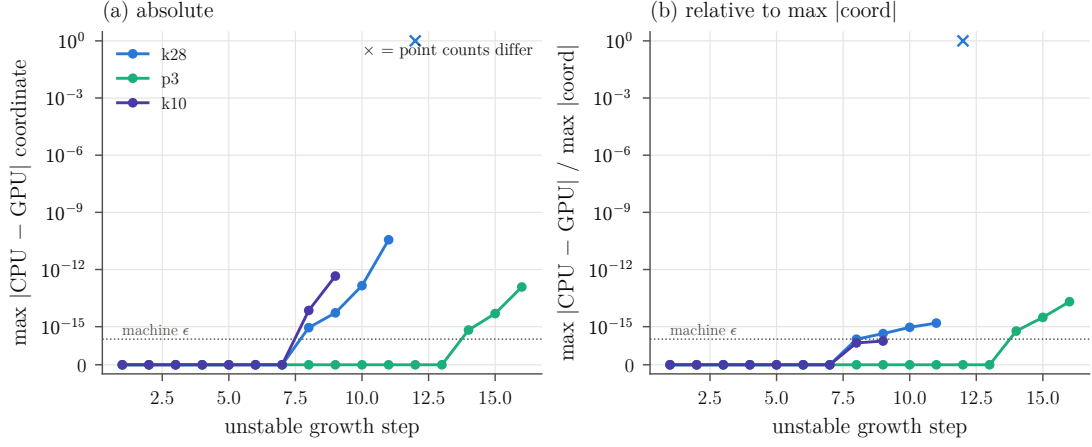
| array    | exact CPU-CPU | exact GPU-GPU | exact CPU-GPU | $10^{-12}$ CPU-CPU | $10^{-12}$ GPU-GPU | $10^{-12}$ CPU-GPU |
|----------|---------------|---------------|---------------|--------------------|--------------------|--------------------|
| unstable | yes           | yes           | <b>no</b>     | yes                | yes                | <b>no</b>          |
| stable   | yes           | yes           | <b>no</b>     | yes                | yes                | <b>no</b>          |
| crossing | yes           | yes           | <b>no</b>     | yes                | yes                | yes                |
| cdist    | yes           | yes           | <b>no</b>     | yes                | yes                | yes                |
| bridge   | yes           | yes           | <b>no</b>     | yes                | yes                | <b>no</b>          |



**Figure 23:** E10. (a) Stage times over five repeats, labelled with their CV: the spread is a few percent. (b) Total time per repeat: no drift, and the GPU is consistently slower on this mid-size case.

**Interpretation.** The CPU-GPU differences are rounding differences in the map that scale with the coordinate magnitude. In the lockstep comparison (Figure 24) they first appear at step 8. At step 11 of  $k28$  they reach  $3.6 \times 10^{-11}$  in absolute terms but only  $1.5 \times 10^{-15}$  relative to the largest coordinate. At step 12, with coordinates near  $5.5 \times 10^8$ , they flip refinement decisions: the CPU holds 4348557 points and the GPU 4348603. The crossings, which live near the saddle, are unaffected.

**Caveat.** Five repeats bound the CV loosely. A timing difference below  $\sim 5\%$  between two configurations should not be read as real.



**Figure 24:** E2 lockstep CPU/GPU deviation of the unstable manifolds. (a) Absolute: it grows once the arm escapes. (b) Relative to the largest coordinate: it stays at machine  $\epsilon$ . The cross marks the step where the point counts differ.

## 12 Failure taxonomy

Table 7 collects every failure class the campaign hit. “By design” means the library refuses the input on purpose; “limitation” means correct behaviour within an unstated envelope; “bug” means behaviour the library’s own contract rules out.

**Table 7:** Failure taxonomy.

| failure class                 | trigger                                                                              | symptom                                        | kind                         | suggested fix                                                                       |
|-------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------|------------------------------|-------------------------------------------------------------------------------------|
| Escape OOM                    | fixed step count past the tangle (Sections 3 and 9.1)                                | one step goes from MB to the RSS cap; OOM kill | limitation                   | point/memory budget in <code>grow_n_times</code> ; stop-condition growth by default |
| Inversion blow-up             | inversion saddle at 7 and 8 unstable steps; $k \geq 10.2$ in E6                      | OOM, documented as “does not finish”           | limitation                   | same budget guard                                                                   |
| Artefact crossings            | near tangency, $k + 1 < 0.16$ (Section 9.4)                                          | crossing count set by cutoff; no warning       | limitation (silent)          | angle/invariance-error guard; resolution warning                                    |
| Self-crossing polygon         | coarse cutoff, e.g. $10^{-5}$ (Section 7)                                            | $u \times u$ crossings undetected              | limitation (silent)          | cheap $u \times u$ diagnostic                                                       |
| Coarse-cutoff crossing errors | default $10^{-4}$ on p3 (Section 5)                                                  | crossings invented and missed                  | limitation                   | default $10^{-7}$                                                                   |
| Wrong period                  | <code>compute_fixed_point</code> from an arbitrary seed (Section 10)                 | lower-period orbit accepted as period $p$      | bug                          | minimal-period check in <code>construct_fixed_point</code>                          |
| Max iterations                | $k \leq 0.545$ , <code>grow_until_turnaround</code> (Section 8)                      | <b>ValueError</b>                              | limitation                   | raise cap or change stop condition at small $\lambda_u$                             |
| Trivial tangle                | fixed recipe at small $k$ (Section 8)                                                | completes with 0 crossings                     | limitation                   | scale depth with $1/\ln \lambda_u$                                                  |
| Not a saddle                  | elliptic point                                                                       | <b>ValueError</b>                              | by design                    | none                                                                                |
| Orientation reversing         | $b = -1$                                                                             | <b>ValueError</b>                              | by design                    | none (out of scope)                                                                 |
| Partition assert              | inversion at 4 and 5 steps (Section 9.2)                                             | “owned by 0 partition elements”                | bug or too strict (topology) | topology owner to triage                                                            |
| Unreliable topology           | $\geq 15$ p3 steps; many blasts (Sections 9.2 and 9.3)                               | <code>is_reliable=False</code>                 | limitation (topology)        | known open issue                                                                    |
| Stale registry                | caching <code>intersection_registry</code> across <code>compute_intersections</code> | empty/stale object                             | API trap                     | return or re-fetch; warn in docs                                                    |
| <code>image_of</code> misses  | 39 of the regions tried (Section 6)                                                  | no image found                                 | open question                | investigate                                                                         |

## 13 Recommendations

### Defaults.

- **area\_cutoff:**  $10^{-7}$ . It is converged in every case tested, and its cost over  $10^{-4}$  is the point ratio  $450 \rightarrow 4757$  on k28 (Section 5).
- **Growth:** stop-condition growth (`grow_until_*`) as the documented default. `grow_n_times` only with a budget.
- **GPU:** off. The measured ceiling is  $1.008\times$  (Section 4).

### Guards to add (cheap, numerics layer).

- **Self-crossing diagnostic:** test non-adjacent  $u \times u$  (and  $s \times s$ ) segment pairs through the existing R-tree after growth, and warn with the cutoff that would remove them.
- **Angle/resolution warning:** for each registered crossing, compare  $|\sin \theta|$  times the local segment length with the local invariance error, and warn when the crossing cannot be distinguished from polygon noise.

- **Minimal-period check:** in `construct_fixed_point`, reject (or relabel) an orbit whose minimal period is a proper divisor of the requested one.
- **Memory guard:** predict the next step’s points from the last growth factor, and refuse or warn above a budget. The E3 harness already does this in a few lines.

#### Open questions for the advisor.

1. Near tangency, should we accept that crossings below a resolution threshold are undecidable with polygons, or invest in a validated method (interval arithmetic, or parameterisation-method manifolds)?
2. Is `image_of` expected to fail for regions whose image crossings are not registered (39 of the regions tried), or is this a gap?
3. Should dissipative maps ( $b \neq \pm 1$ ) be in scope? They currently run without complaint and without validation.
4. Is the inversion partition assertion at shallow depth a real defect or an over-strict invariant?

## 14 Threats to validity

- **One machine.** All timings are from a single host and one GPU. The GPU conclusion rests on Amdahl’s law, which transfers to other hardware; the absolute crossover sizes do not.
- **Hénon only.** Every result is for the Hénon family. Maps with more expensive evaluations would raise  $f_{\text{map}}$  and could change the GPU verdict.
- **Quick calibration of grids.** The sweep grids and depths were calibrated in a short pilot to fit the budget. Exponents rest on  $\leq 3$  decades, and some breaking points (depth, blasts) are only known to the resolution of the grid.
- **cProfile overhead.** Profiling inflates Python call costs relative to NumPy kernels, which biases  $f_{\text{map}}$  low (see the caveat in Section 4).
- **RSS sampling.** The watchdog samples every 50 ms, so peak RSS can be missed by a short spike, and OOM kills happen up to one sample after the cap. Caps differ per experiment (Table 10).
- **Invariance probe design.** The probe maps points sampled inside segments, because a vertex’s image is often exactly a vertex, and measures one-step consistency. It is not a distance to the true manifold.
- **References are finest runs.** Convergence in E4 is measured against  $10^{-9}$ , and E9’s “truth” is a brute force on the *same* polygons. E9 therefore validates the intersection code, not the manifold approximation.

## A Tables

**Table 8:** E4: cost and accuracy per cutoff. “crossing dev.” is the median distance of the finest run’s crossings to this run’s.

| case | steps | cutoff    | outcome | points    | time (s)             | inv. p50             | inv. p99             | crossings | crossing dev.        |
|------|-------|-----------|---------|-----------|----------------------|----------------------|----------------------|-----------|----------------------|
| k28  | 10    | $10^{-3}$ | ok      | 222       | $9.9 \times 10^{-3}$ | $3.7 \times 10^{-4}$ | $5.6 \times 10^{-3}$ | 5         | $1.7 \times 10^{-3}$ |
| k28  | 10    | $10^{-4}$ | ok      | 450       | 0.0148               | $1.3 \times 10^{-4}$ | $9.6 \times 10^{-4}$ | 5         | $2.2 \times 10^{-4}$ |
| k28  | 10    | $10^{-5}$ | ok      | 1 029     | 0.0274               | $3.2 \times 10^{-5}$ | $2.2 \times 10^{-4}$ | 5         | $7.7 \times 10^{-5}$ |
| k28  | 10    | $10^{-6}$ | ok      | 2 194     | 0.0511               | $1.0 \times 10^{-5}$ | $4.8 \times 10^{-5}$ | 5         | $1.7 \times 10^{-5}$ |
| k28  | 10    | $10^{-7}$ | ok      | 4 757     | 0.107                | $1.8 \times 10^{-6}$ | $6.9 \times 10^{-6}$ | 5         | $2.3 \times 10^{-6}$ |
| k28  | 10    | $10^{-8}$ | ok      | 10 028    | 0.247                | $3.6 \times 10^{-7}$ | $1.7 \times 10^{-6}$ | 5         | $5.6 \times 10^{-7}$ |
| k28  | 10    | $10^{-9}$ | ok      | 20 988    | 0.515                | $7.9 \times 10^{-8}$ | $4.4 \times 10^{-7}$ | 5         | ref.                 |
| k28  | 12    | $10^{-3}$ | ok      | 18 801    | 0.449                | $2.3 \times 10^{-4}$ | $5.6 \times 10^{-3}$ | 17        | $1.2 \times 10^{-3}$ |
| k28  | 12    | $10^{-4}$ | ok      | 40 604    | 1.01                 | $1.3 \times 10^{-4}$ | $7.3 \times 10^{-4}$ | 17        | $2.6 \times 10^{-4}$ |
| k28  | 12    | $10^{-5}$ | ok      | 86 407    | 2.18                 | $3.2 \times 10^{-5}$ | $1.5 \times 10^{-4}$ | 17        | $6.9 \times 10^{-5}$ |
| k28  | 12    | $10^{-6}$ | ok      | 191 411   | 5.22                 | $1.0 \times 10^{-5}$ | $3.4 \times 10^{-5}$ | 17        | $1.3 \times 10^{-5}$ |
| k28  | 12    | $10^{-7}$ | ok      | 408 846   | 11.6                 | $1.8 \times 10^{-6}$ | $6.9 \times 10^{-6}$ | 17        | $2.3 \times 10^{-6}$ |
| k28  | 12    | $10^{-8}$ | ok      | 868 883   | 23.9                 | $3.6 \times 10^{-7}$ | $1.7 \times 10^{-6}$ | 17        | $5.6 \times 10^{-7}$ |
| k28  | 12    | $10^{-9}$ | ok      | 1 875 771 | 54.6                 | $7.9 \times 10^{-8}$ | $4.4 \times 10^{-7}$ | 17        | ref.                 |
| p3   | 13    | $10^{-3}$ | ok      | 127       | 0.0195               | $8.3 \times 10^{-5}$ | $2.7 \times 10^{-3}$ | 14        | 0.11                 |
| p3   | 13    | $10^{-4}$ | ok      | 207       | 0.0179               | $1.1 \times 10^{-4}$ | $1.4 \times 10^{-3}$ | 37        | $6.5 \times 10^{-4}$ |
| p3   | 13    | $10^{-5}$ | ok      | 306       | 0.021                | $1.2 \times 10^{-4}$ | $6.7 \times 10^{-4}$ | 27        | $1.7 \times 10^{-4}$ |
| p3   | 13    | $10^{-6}$ | ok      | 552       | 0.0367               | $3.0 \times 10^{-5}$ | $9.1 \times 10^{-5}$ | 27        | $6.9 \times 10^{-5}$ |
| p3   | 13    | $10^{-7}$ | ok      | 1 127     | 0.0462               | $8.9 \times 10^{-6}$ | $1.9 \times 10^{-5}$ | 27        | $1.2 \times 10^{-5}$ |
| p3   | 13    | $10^{-8}$ | ok      | 2 451     | 0.0932               | $1.7 \times 10^{-6}$ | $4.4 \times 10^{-6}$ | 27        | $2.0 \times 10^{-6}$ |
| p3   | 13    | $10^{-9}$ | ok      | 5 485     | 0.175                | $3.9 \times 10^{-7}$ | $9.0 \times 10^{-7}$ | 27        | ref.                 |
| p3   | 17    | $10^{-3}$ | oom     | n/a       | n/a                  | n/a                  | n/a                  | n/a       | n/a                  |
| p3   | 17    | $10^{-4}$ | ok      | 43 340    | 1.09                 | $3.0 \times 10^{-4}$ | $1.4 \times 10^{-3}$ | 249       | $1.4 \times 10^{-3}$ |
| p3   | 17    | $10^{-5}$ | ok      | 117 917   | 3.14                 | $1.0 \times 10^{-4}$ | $4.0 \times 10^{-4}$ | 214       | $6.0 \times 10^{-4}$ |
| p3   | 17    | $10^{-6}$ | ok      | 276 531   | 7.49                 | $2.2 \times 10^{-5}$ | $9.7 \times 10^{-5}$ | 213       | $2.1 \times 10^{-4}$ |
| p3   | 17    | $10^{-7}$ | ok      | 585 323   | 16.5                 | $4.9 \times 10^{-6}$ | $2.0 \times 10^{-5}$ | 203       | $5.4 \times 10^{-5}$ |
| p3   | 17    | $10^{-8}$ | ok      | 1 278 043 | 36.7                 | $1.0 \times 10^{-6}$ | $3.9 \times 10^{-6}$ | 203       | $1.1 \times 10^{-5}$ |
| p3   | 17    | $10^{-9}$ | ok      | 2 677 865 | 77.7                 | $2.2 \times 10^{-7}$ | $8.7 \times 10^{-7}$ | 203       | ref.                 |

**Table 9:** E8: solver outcomes per map and requested period. “orbits found” counts distinct orbits of exactly that minimal period over all solves (for  $k = 10$ : found / full 2-shift count).

| map       | period | converged | saddle | lower period | orbits found | max residual          |
|-----------|--------|-----------|--------|--------------|--------------|-----------------------|
| $k = 10$  | 1      | 100%      | 100%   | 0            | 2/2          | 0                     |
| $k = 10$  | 2      | 100%      | 50.2%  | 49.8%        | 1/1          | $5.8 \times 10^{-14}$ |
| $k = 10$  | 3      | 99.8%     | 75%    | 24.8%        | 2/2          | $8.2 \times 10^{-13}$ |
| $k = 10$  | 4      | 97.8%     | 75.5%  | 22.8%        | 3/3          | $5.8 \times 10^{-11}$ |
| $k = 10$  | 5      | 83%       | 79%    | 4.82%        | 6/6          | $4.7 \times 10^{-12}$ |
| $k = 10$  | 6      | 62%       | 53.5%  | 13.7%        | 9/9          | $1.8 \times 10^{-11}$ |
| $k = 2$   | 1      | 85.8%     | 36.5%  | 0            | 2            | $8.9 \times 10^{-16}$ |
| $k = 2$   | 2      | 99.8%     | 0      | 100%         | 0            | $6.2 \times 10^{-15}$ |
| $k = 2$   | 3      | 100%      | 73.2%  | 26.8%        | 2            | $1.1 \times 10^{-13}$ |
| $k = 2$   | 4      | 98.8%     | 60.5%  | 38.7%        | 2            | $6.6 \times 10^{-13}$ |
| $k = 2$   | 5      | 73.5%     | 44.2%  | 39.8%        | 2            | $7.6 \times 10^{-13}$ |
| $k = 2$   | 6      | 66.5%     | 45%    | 32.3%        | 3            | $3.5 \times 10^{-12}$ |
| $k = 2.1$ | 1      | 100%      | 30%    | 0            | 2            | $1.3 \times 10^{-15}$ |
| $k = 2.1$ | 2      | 100%      | 0      | 100%         | 0            | $2.7 \times 10^{-15}$ |
| $k = 2.1$ | 3      | 100%      | 79.2%  | 20.8%        | 2            | $2.8 \times 10^{-13}$ |
| $k = 2.1$ | 4      | 99.5%     | 56.8%  | 43%          | 2            | $8.3 \times 10^{-13}$ |
| $k = 2.1$ | 5      | 72.5%     | 44.2%  | 39%          | 2            | $1.3 \times 10^{-12}$ |
| $k = 2.1$ | 6      | 61.8%     | 39.5%  | 36%          | 3            | $1.6 \times 10^{-12}$ |
| $k = 2.8$ | 1      | 83%       | 31%    | 0            | 2            | $8.9 \times 10^{-16}$ |
| $k = 2.8$ | 2      | 99.8%     | 0      | 100%         | 0            | $1.7 \times 10^{-14}$ |
| $k = 2.8$ | 3      | 99.8%     | 73%    | 26.8%        | 2            | $2.2 \times 10^{-13}$ |
| $k = 2.8$ | 4      | 99.5%     | 61%    | 38.7%        | 2            | $1.7 \times 10^{-12}$ |
| $k = 2.8$ | 5      | 67.8%     | 48.2%  | 9.59%        | 4            | $4.1 \times 10^{-12}$ |
| $k = 2.8$ | 6      | 62.3%     | 37%    | 40.6%        | 3            | $1.5 \times 10^{-11}$ |

## B Reproduction

From the repository root, with the project’s environment:

```
env/bin/python stress/numerics/run_campaign.py --quick # pilot
env/bin/python stress/numerics/run_campaign.py # full run
env/bin/python stress/numerics/report/make_figures.py
env/bin/python stress/numerics/report/make_numbers.py
env/bin/python stress/numerics/report/make_numbers_extra.py
cd stress/numerics/report && ~/.local/bin/tectonic report.tex
```

Results land in `stress/numerics/results/*.jsonl` (one record per configuration) with a running `progress.log`. A single experiment runs on its own as

```
env/bin/python stress/numerics/eNN_*.py [--quick] [--only I,J] \
  [--rss-cap-gb G] [--timeout S] [--workers N]
```

## C Configuration and outcome counts

**Table 10:** Records and outcomes per experiment in the full run, with each experiment’s default per-configuration timeout and RSS cap (some E7 configs override the timeout). “assert” counts `invariant_assert` outcomes of the numerics; the topology assertions of E7 are secondary results inside `ok` records.

| experiment             | configs | ok  | exception | assert | oom | timeout | timeout (s) | cap (GB) |
|------------------------|---------|-----|-----------|--------|-----|---------|-------------|----------|
| e01_map_throughput     | 8       | 8   | 0         | 0      | 0   | 0       | 300         | 24       |
| e02_gpu_end_to_end     | 33      | 33  | 0         | 0      | 0   | 0       | 600         | 24       |
| e03_scaling            | 22      | 19  | 0         | 0      | 3   | 0       | 900         | 24       |
| e04_area_cutoff        | 28      | 27  | 0         | 0      | 1   | 0       | 900         | 16       |
| e05_invariants         | 8       | 8   | 0         | 0      | 0   | 0       | 600         | 12       |
| e06_parameter_sweep    | 98      | 71  | 24        | 0      | 3   | 0       | 240         | 4        |
| e07_breaking           | 92      | 88  | 0         | 0      | 4   | 0       | 600         | 20       |
| e08_fixed_point_solver | 24      | 24  | 0         | 0      | 0   | 0       | 900         | 24       |
| e09_intersections      | 26      | 26  | 0         | 0      | 0   | 0       | 600         | 16       |
| e10_determinism        | 10      | 10  | 0         | 0      | 0   | 0       | 300         | 24       |
| total                  | 349     | 314 | 24        | 0      | 11  | 0       |             |          |